



Software Requirements Specification

Lost Cat Finder · AI Vision Matching · Rescue Relay Network

HackTheKitty Hackathon 2026 | World Cat Domination Day Theme

Version 1.0 | June 24, 2026

React · Node.js · Express · PostgreSQL · Claude Vision API · Socket.io · Leaflet.js

Project Name	PawTrace — Lost Cat Finder & Rescue Relay
Document Type	Software Requirements Specification (SRS)
Version	1.0
Hackathon	HackTheKitty 2026 — World Cat Domination Day
Duration	June 24 – July 7, 2026 (14 days)
Team Size	1–2 Members
Prepared By	Team PawTrace
Date	June 24, 2026

■ Table of Contents

1.	Introduction	3
2.	Overall Description	3
3.	Tech Stack & Architecture	4
4.	Database Schema (PostgreSQL)	5
5.	Functional Requirements	6
6.	Non-Functional Requirements	8
7.	API Endpoints	8
8.	UI/UX Requirements	9
9.	Security Requirements	10
10.	Project Timeline	10
11.	Judging Criteria Mapping	11
12.	Assumptions & Constraints	11

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete requirements for **PawTrace**, a community-driven web application built for the HackTheKitty 2026 hackathon under the *World Cat Domination Day* theme. The document covers functional requirements, system architecture, database design, API contracts, and non-functional constraints to guide development within the 14-day window.

1.2 Scope

PawTrace addresses three core problems faced by cat owners and the broader cat rescue community:

- **Lost/Found Reporting:** A real-time Leaflet.js map where users post lost or found cat sightings with geo-tagged pins.
- **AI Vision Matching:** Claude Vision API automatically extracts visual features from uploaded cat photos and matches new reports against existing ones, returning a confidence score.
- **Rescue Relay Network:** A volunteer coordination layer using Socket.io where spotters who cannot catch a cat can broadcast requests to nearby registered volunteers.

1.3 Definitions & Acronyms

SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
JWT	JSON Web Token — used for stateless authentication
PGSQL	PostgreSQL — relational database system
TNR	Trap-Neuter-Return — community cat management practice
AI Match	Claude Vision API feature that matches cat photos by visual features
Relay	Volunteer coordination system for rescuing spotted cats

2. Overall Description

2.1 Product Perspective

PawTrace is a standalone web application with a React SPA frontend, Node.js/Express REST backend, PostgreSQL database, and real-time Socket.io layer. It integrates the Anthropic Claude API for vision-based cat photo analysis and OpenStreetMap/Leaflet for mapping — both free-tier friendly for hackathon use.

2.2 User Classes

Guest	Browse the map, view lost/found pins without login
Cat Owner	Register, post lost/found reports, upload photos, receive match alerts
Volunteer	Register as a rescue volunteer, receive relay pings, update relay status

2.3 Operating Environment

- Modern web browsers (Chrome 90+, Firefox 88+, Safari 14+)
- Node.js v20+ runtime on server
- PostgreSQL 15+ database
- Deployment: any VPS / cloud provider (Railway, Render, Vercel + Supabase)

3. Tech Stack & Architecture

3.1 Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite	SPA framework
Styling	Tailwind CSS	Utility-first styling
Maps	Leaflet.js + OpenStreetMap	Free geo mapping
Backend	Node.js + Express	REST API server
Auth	JWT (jsonwebtoken)	Stateless authentication
Database	PostgreSQL 15	Relational data store
ORM	Prisma	Type-safe DB queries
Realtime	Socket.io	Volunteer relay events
AI Vision	Anthropic Claude API	Cat photo feature extraction & matching
File Upload	Multer + Cloudinary	Image storage
Email	Nodemailer / Resend	Match & relay notifications

3.2 Why Node.js over Python for AI?

The Claude Vision API is a standard HTTPS REST endpoint. Any language capable of making HTTP requests can call it. Node.js is recommended for this project because:

- Both team members have stronger JavaScript knowledge than Python
- Single language across backend eliminates context switching
- Socket.io is natively Node.js — no bridge needed
- Claude API call is one `fetch()` with a base64 image payload — no Python ML libraries required

Note: Python would only be beneficial if running a local ML model (e.g. PyTorch). Since we are using Claude's hosted API, Node.js is equally capable and significantly faster to develop in.

3.3 High-Level Architecture

The system follows a standard 3-tier architecture:

Client (React SPA)	↔	Server (Express REST + Socket.io)	↔	PostgreSQL
Leaflet Map, Report Forms, Match Results, Relay Dashboard		Auth, Reports, AI Matching, Volunteer Relay, Notifications		Users, Reports, Matches, Volunteers, Relay Events

■ 4. Database Schema (PostgreSQL)

All tables use UUID primary keys and include created_at / updated_at timestamps. The schema is managed via Prisma migrations.

4.1 users

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique user ID
name	VARCHAR(100)	NOT NULL	Full name
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email
password_hash	TEXT	NOT NULL	bcrypt hashed password
role	ENUM('owner','volunteer','admin')	DEFAULT 'owner'	User role
location_lat	DECIMAL(9,6)	NULLABLE	User's home latitude
location_lng	DECIMAL(9,6)	NULLABLE	User's home longitude
notify_radius_km	INTEGER	DEFAULT 5	Alert radius in km
created_at	TIMESTAMP	DEFAULT NOW()	Registration time

4.2 reports

Column	Type	Constraints	Description
id	UUID	PK	Report ID
user_id	UUID	FK → users.id	Reporter
type	ENUM('lost','found')	NOT NULL	Report type
status	ENUM('active','reunited','closed')	DEFAULT 'active'	Current status
cat_name	VARCHAR(100)	NULLABLE	Cat's name if known
description	TEXT	NOT NULL	Physical description
photo_url	TEXT	NOT NULL	Cloudinary image URL
ai_features	JSONB	NULLABLE	Extracted AI features
lat	DECIMAL(9,6)	NOT NULL	Sighting latitude
lng	DECIMAL(9,6)	NOT NULL	Sighting longitude
created_at	TIMESTAMP	DEFAULT NOW()	Report time

4.3 matches

Column	Type	Constraints	Description
id	UUID	PK	Match ID
report_a_id	UUID	FK → reports.id	Lost report
report_b_id	UUID	FK → reports.id	Found report

Column	Type	Constraints	Description
confidence	DECIMAL(5,2)	NOT NULL	AI match score 0–100
notified	BOOLEAN	DEFAULT false	Alert sent?
created_at	TIMESTAMP	DEFAULT NOW()	Match time

4.4 volunteers

Column	Type	Constraints	Description
id	UUID	PK	Record ID
user_id	UUID	FK → users.id, UNIQUE	Linked user
can_foster	BOOLEAN	DEFAULT false	Temporary fostering?
active	BOOLEAN	DEFAULT true	Currently available?
radius_km	INTEGER	DEFAULT 3	Service radius

4.5 relay_events

Column	Type	Constraints	Description
id	UUID	PK	Event ID
report_id	UUID	FK → reports.id	Related report
spotter_id	UUID	FK → users.id	Who spotted the cat
volunteer_id	UUID	FK → users.id, NULLABLE	Assigned volunteer
status	ENUM('pending','accepted','caught','fostering','reunited')	DEFAULT 'pending'	Relay chain status
notes	TEXT	NULLABLE	Volunteer notes
created_at	TIMESTAMP	DEFAULT NOW()	Event time
updated_at	TIMESTAMP	DEFAULT NOW()	Last update

5. Functional Requirements

5.1 Authentication Module

ID	Requirement	Priority	Module
FR-01	User can register with name, email, password	High	Auth
FR-02	Password stored as bcrypt hash (salt rounds ≥ 12)	High	Auth
FR-03	User can login and receive JWT access token (15min) + refresh token (7d)	High	Auth
FR-04	Protected routes validate JWT via Authorization header	High	Auth
FR-05	User can update profile (name, location, notification radius)	Med	Auth

5.2 Report Module (1A — Lost/Found Map)

ID	Requirement	Priority	Module
FR-06	Authenticated user can create a Lost or Found report with photo, description, location (lat/lng)	High	Reports
FR-07	Map displays all active reports as color-coded pins (red=lost, green=found)	High	Reports
FR-08	Guest users can browse the map without logging in	High	Reports
FR-09	User can click a pin to view full report details (photo, description, contact)	High	Reports
FR-10	Reporter can update status to 'Reunited' or 'Closed'	High	Reports
FR-11	System sends in-app alert to users within configured radius when new report is posted	High	Reports
FR-12	Reports can be filtered by type (lost/found), date, and distance	Med	Reports
FR-13	Reporter can edit description and photo of their own reports	Med	Reports

5.3 AI Vision Matching Module (1B)

ID	Requirement	Priority	Module
FR-14	On photo upload, system sends image to Claude Vision API with a structured feature extraction prompt	High	AI Match
FR-15	AI extracts: fur color, pattern (tabby/solid/calico etc.), eye color, distinctive markings, estimated breed, size	High	AI Match
FR-16	Extracted features stored as JSONB in reports.ai_features	High	AI Match
FR-17	On new report creation, system compares features against all opposite-type active reports within 20km	High	AI Match
FR-18	Matches with confidence $\geq 50\%$ are stored in matches table	High	AI Match

ID	Requirement	Priority	Module
FR-19	Top 3 matches displayed to reporter with confidence percentage and side-by-side photo comparison	High	AI Match
FR-20	Both reporters notified via email when a high-confidence match ($\geq 75\%$) is found	High	AI Match
FR-21	User can dismiss a match as 'Not my cat'	Med	AI Match

5.4 Rescue Relay Module (1C)

ID	Requirement	Priority	Module
FR-22	User can click 'I spotted it but can't catch' on any lost report to trigger a relay event	High	Relay
FR-23	System broadcasts Socket.io event to all active volunteers within 3km of the sighting	High	Relay
FR-24	Volunteer receives real-time ping with cat details and location	High	Relay
FR-25	Volunteer can Accept or Decline the relay request	High	Relay
FR-26	Relay status chain: Pending → Accepted → Caught → Fostering → Reunited	High	Relay
FR-27	Relay status visible in real-time to the original reporter	High	Relay
FR-28	User can register as a volunteer by enabling 'Volunteer Mode' in profile	High	Relay
FR-29	Volunteer can toggle their active status (on/off) and set service radius	Med	Relay
FR-30	Volunteer can mark themselves as able to foster temporarily	Med	Relay

6. Non-Functional Requirements

ID	Requirement	Priority	Category
NFR-01	API response time < 500ms for all non-AI endpoints under normal load	High	Performance
NFR-02	AI matching response delivered within 5 seconds of report submission	High	Performance
NFR-03	Application usable on mobile screens (responsive at 375px+)	High	UX/UI
NFR-04	All passwords hashed with bcrypt, never stored in plaintext	High	Security
NFR-05	JWT tokens validated on every protected request; tokens not stored in localStorage (use httpOnly cookies)	High	Security
NFR-06	All API inputs sanitized and validated (express-validator)	High	Security
NFR-07	File uploads restricted to image types only (jpg/png/webp, max 5MB)	High	Security
NFR-08	Rate limiting on auth endpoints (max 5 login attempts / 15 min per IP)	Med	Security
NFR-09	README covers setup, env variables, API docs, and project overview	High	Docs
NFR-10	All API endpoints documented with request/response examples	High	Docs

7. API Endpoints

7.x Auth

Method	Endpoint	Description
POST	/api/auth/register	Register new user
POST	/api/auth/login	Login, returns JWT
POST	/api/auth/refresh	Refresh access token
GET	/api/auth/me	Get current user profile (protected)
PUT	/api/auth/me	Update profile (protected)

7.x Reports

Method	Endpoint	Description
GET	/api/reports	Get all active reports (with filters)
POST	/api/reports	Create new report + trigger AI match (protected)
GET	/api/reports/:id	Get single report details
PUT	/api/reports/:id	Update report (owner only)
DELETE	/api/reports/:id	Delete report (owner only)

Method	Endpoint	Description
GET	/api/reports/nearby	Get reports within radius of lat/lng

7.x Matches

Method	Endpoint	Description
GET	/api/matches/:reportId	Get matches for a report
PUT	/api/matches/:id/dismiss	Dismiss a match as 'Not my cat'

7.x Relay

Method	Endpoint	Description
POST	/api/relay	Trigger relay for a report (protected)
GET	/api/relay/:id	Get relay event details
PUT	/api/relay/:id/respond	Volunteer accepts/declines (protected)
PUT	/api/relay/:id/status	Update relay chain status (volunteer)

7.x Volunteer

Method	Endpoint	Description
POST	/api/volunteer/register	Register as volunteer (protected)
PUT	/api/volunteer/toggle	Toggle active status (protected)

8. UI/UX Requirements

8.1 Key Screens

Screen	Key Elements
Home / Map	Full-screen Leaflet map, floating filter bar, colored pins, geolocation button, 'Report Cat' FAB
Report Form	Type toggle (Lost/Found), photo upload with preview, description field, location picker on map, submit
Match Results	Side-by-side photo comparison, confidence bar, feature diff list, 'This is my cat!' / 'Not my cat' buttons
Relay Dashboard	Active relay events near me, status chain timeline, Accept/Decline buttons, live status updates
Volunteer Profile	Toggle volunteer mode, set radius, foster toggle, active status switch
Report Detail	Full photo, description, reporter contact, 'Trigger Relay' button, match suggestions

8.2 Design Guidelines

- Dark-themed UI preferred (cat-night aesthetic) with orange accent color (#E8834A)
- Map-first layout — map occupies full viewport on home screen
- Mobile-first responsive design using Tailwind breakpoints
- Loading skeleton states for map pins and AI match results
- Toast notifications for relay pings and match alerts
- Accessible color contrast ratios (WCAG AA minimum)

9. Security Requirements

Auth	JWT in httpOnly cookies; refresh token rotation; bcrypt for passwords
Input	express-validator on all body params; Parameterized queries via Prisma (no raw SQL)
File Upload	Mime-type whitelist; max 5MB; virus scan via Cloudinary
Rate Limiting	express-rate-limit on /auth/* endpoints (5 req/15min per IP)
CORS	Restrict to frontend origin only; no wildcard in production
Env Vars	API keys in .env; never committed to git; .env.example provided
HTTPS	TLS enforced in production deployment

10. Project Timeline (14 Days)

Days	Tasks	Deliverable
1–2	Repo setup, DB schema (Prisma + PGSQL), JWT auth routes, basic Express API skeleton	Express API skeleton
3–4	Report CRUD API, Leaflet map frontend, colored pins, geolocation	Map with reports
5–6	Nearby alert system, radius filtering, report detail page	Alerts working
7–8	Claude Vision API integration, feature extraction, match scoring algorithm	AI matching live
9–10	Match results UI, side-by-side comparison, email notifications	Match UI done
11–12	Socket.io relay system, volunteer registration, relay dashboard	Relay working
13	Mobile responsiveness, UI polish, error states, loading skeletons	Polished UI
14	README, API docs, env setup guide, deployment, final submission	Submitted ✓

11. Judging Criteria Mapping

Criterion	How PawTrace Addresses It
Technical	Node.js + Express REST API, PostgreSQL with relational schema, Socket.io real-time, JWT auth, Cloudinary
Innovation	AI Vision matching for cats using Claude API — novel application of vision AI for community rescue
Theme Relevance	Directly helps lost cats get home; supports the broader cat rescue community; TNR and volunteer network
Documentation	Full README, API endpoint docs with examples, .env.example, setup guide, this SRS
Security	JWT httpOnly cookies, bcrypt, parameterized queries, rate limiting, input validation, CORS config
UX/UI	Map-first mobile-responsive design, real-time relay updates, match comparison UI, dark cat-themed aesthetics

12. Assumptions & Constraints

Assumptions:

- Users have access to a modern browser with geolocation support
- Anthropic Claude API free tier is sufficient for hackathon-scale photo matching
- Cloudinary free tier (25GB) is adequate for image storage during hackathon
- Deployment will be on a free-tier cloud platform (Render, Railway, or similar)

Constraints:

- All code must be written during June 24 – July 7, 2026 (no pre-written code)
- Team size: maximum 2 members
- 14-day development window limits scope to core features only
- Admin panel and advanced analytics are out of scope for v1
- No native mobile app — PWA considerations only if time permits

